

Sistemas de Informação | AMF
Inteligência Artificial II
19 de agosto de 2026

Aula 3 - Modelos clássicos de ML

Rhauani Fazul

rhauani.fazul@amf.edu.br

Vamos fazer um teste...

VendasXPT0.csv

- Nos próximos 7 dias, qual a previsão de lucro da empresa com base no histórico?
- Nos próximos 7 dias, qual o estado deve gerar mais faturamento e qual faturamento previsto? Qual categoria e qual faturamento previsto?

E com essas informações adicionais?

- A coluna Data do Pedido está no formato mês/dia/ano (MM/DD/AAAA).
- Quando não houver registros em determinado dia, o valor diário deverá ser preenchido com zero.
- A granularidade da análise deverá considerar como modelo de referência uma média móvel de 7 dias.
- Modelo: Regressão Linear

Vamos testar com mais duas perguntas...

- Aumentar os descontos tende a aumentar ou reduzir o lucro?
- Em geral, qual é o impacto da satisfação dos clientes sobre o lucro?

Reflexão

Eficientes mesmo com **datasets médios ou pequenos**.
Costumam ter ótima **acurácia** em problemas tabulares.
São mais **interpretáveis**.
Exigem **menos custo** computacional.

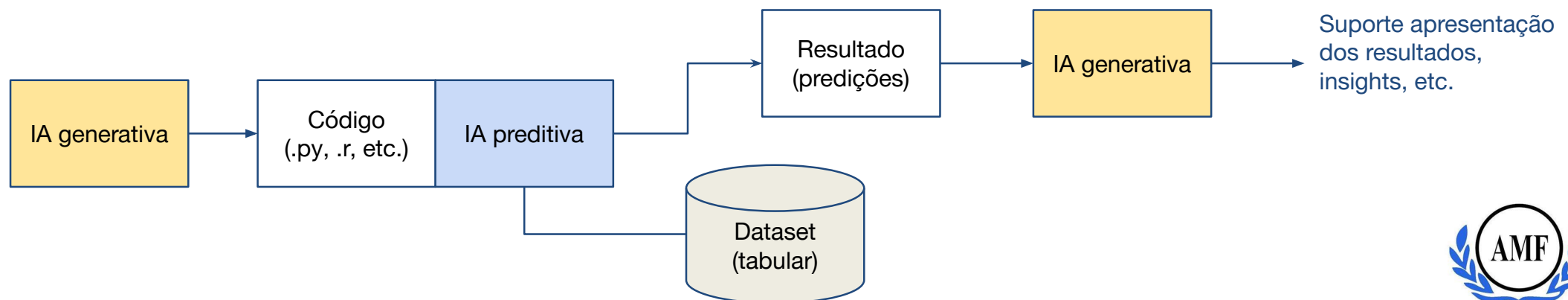
- IA preditiva (ML tradicional)
 - processamento, precisão estatística

- IA generativa
 - orquestradora: interpreta, escolhe, gera, chama ferramentas

Há **datasets gigantescos** (milhões de registros).

Precisa-se capturar **interações muito complexas** entre variáveis.

O problema envolve **multimodalidade**.



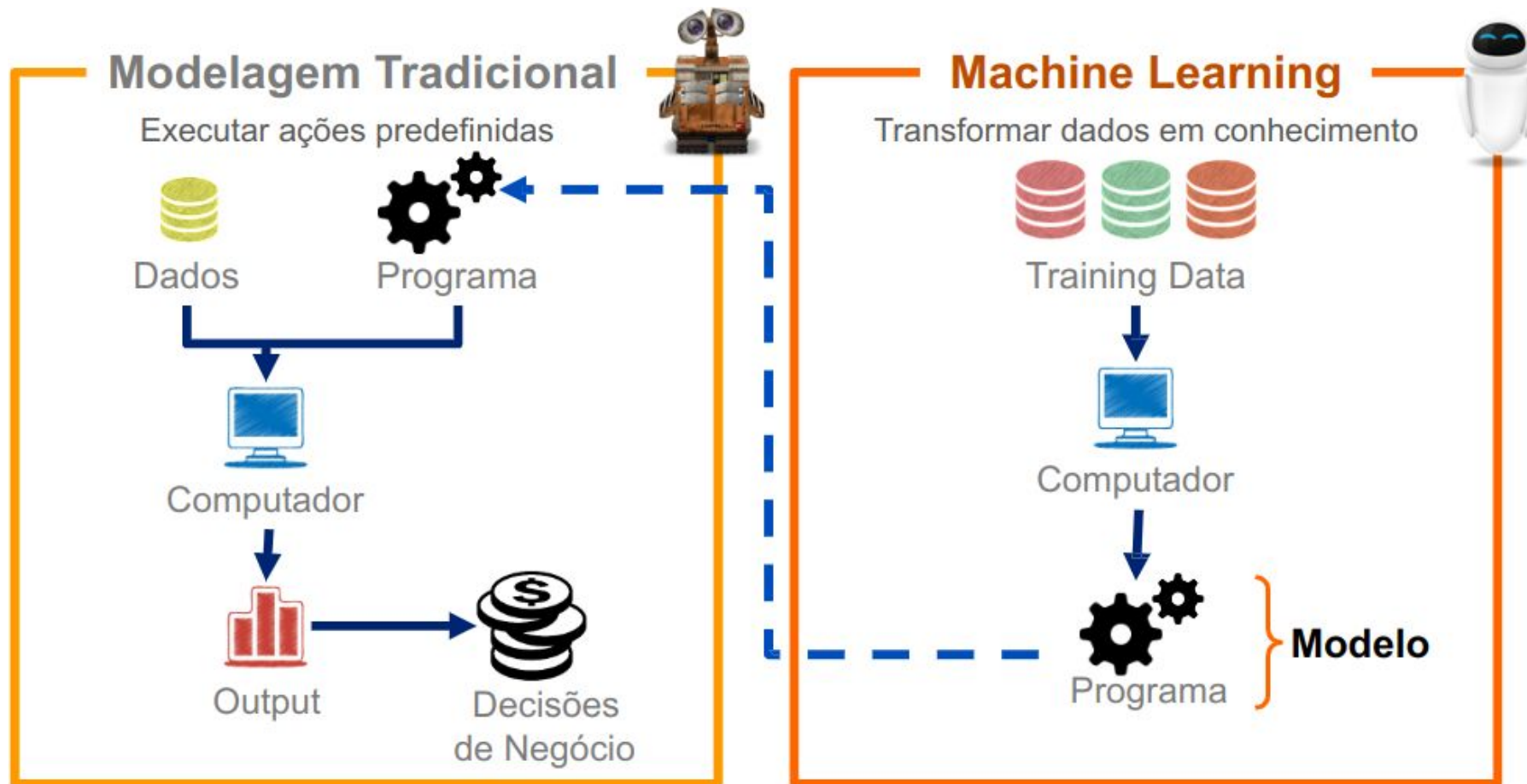
Introdução ao módulo de ML

- O que é aprendizado de máquina / *machine learning*?

Introdução ao módulo de ML

- O que é aprendizado de máquina / *machine learning*?

Relembrando IA I:



Introdução ao módulo de ML

- Qual a diferença entre IA, ML e *Deep Learning*?

Introdução ao módulo de ML

- Qual a diferença entre IA, ML e *Deep Learning*?

Relembrando IA I:

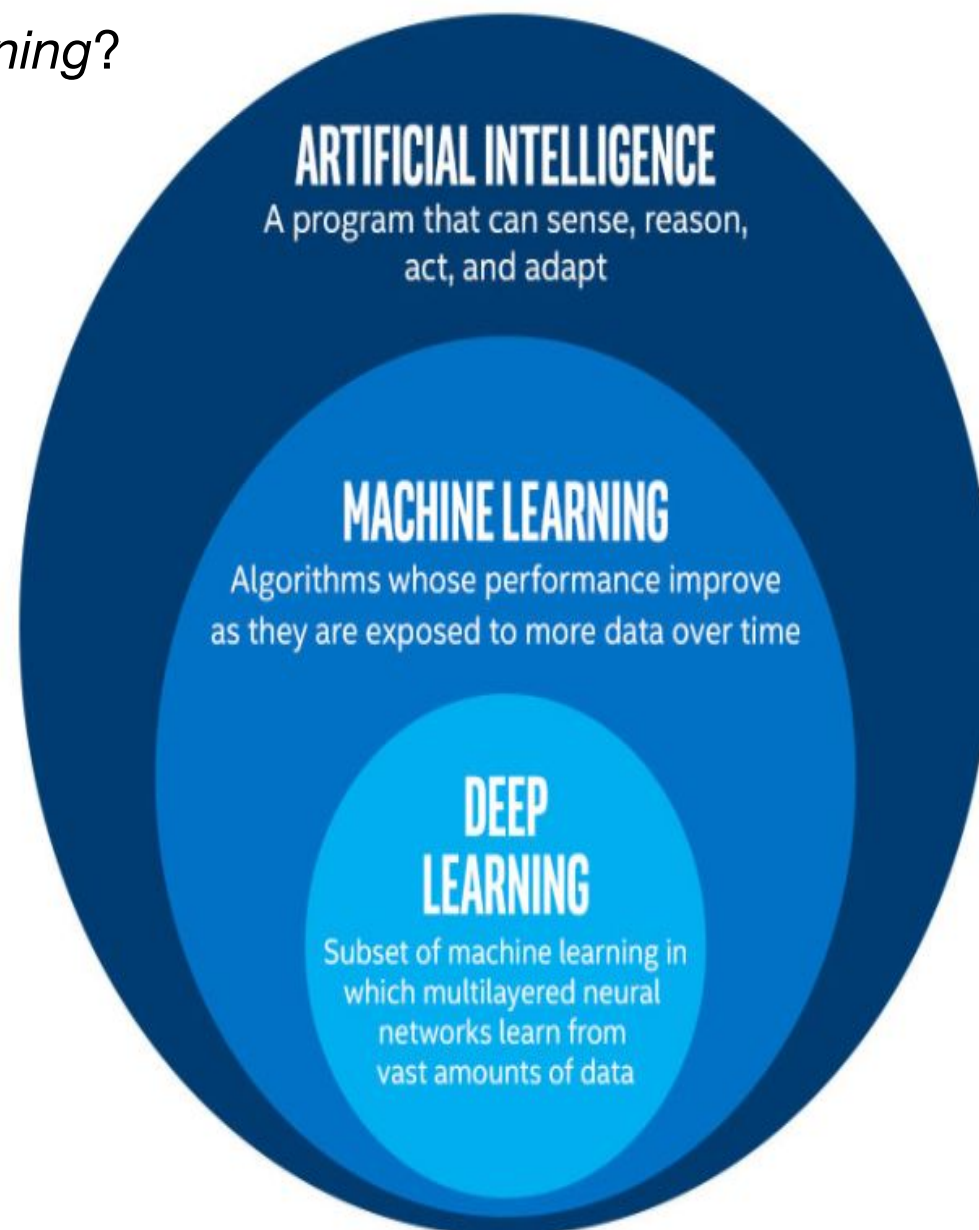
O termo é frequentemente usado de forma intercambiável com seus subcampos.

É uma “novidade” antiga (datada da década de 1950).

“We don’t have better algorithms, we just have more data.”

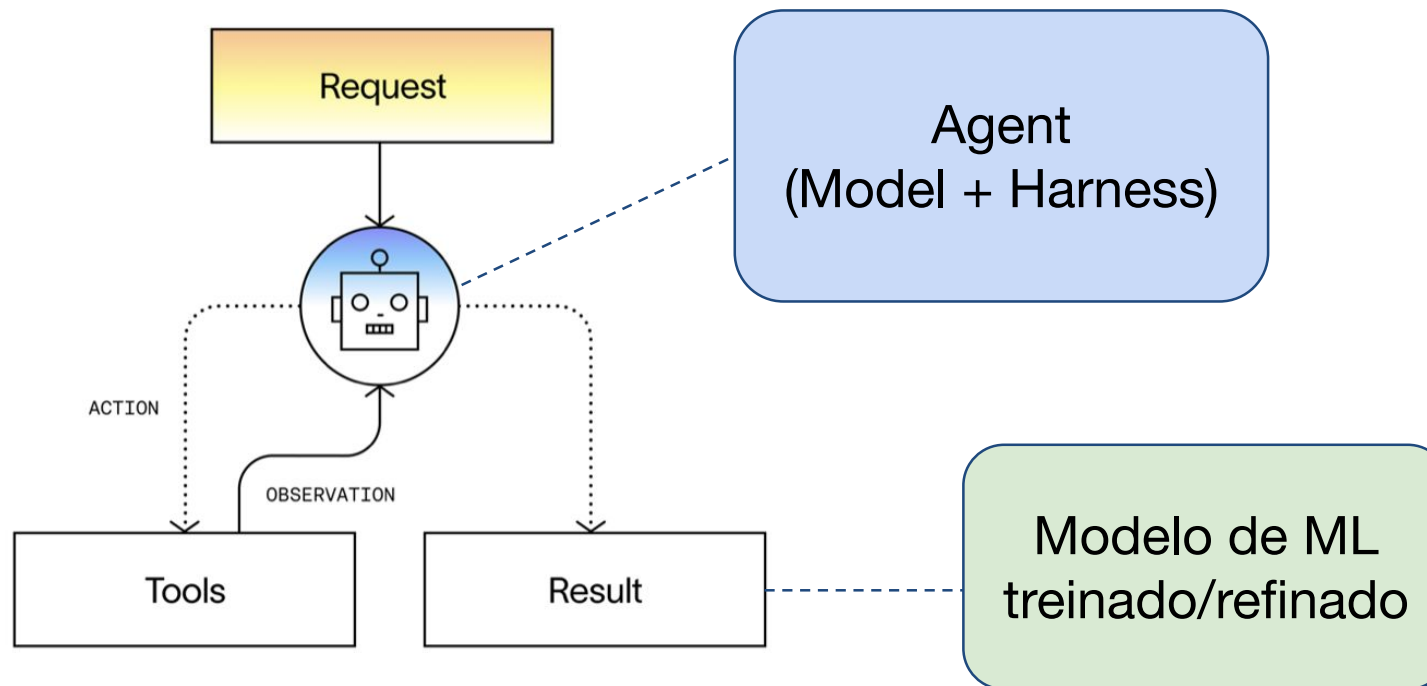
“More data beats clever algorithms, but better data beats more data.”

(Peter Norvig, Director of Research, Google)



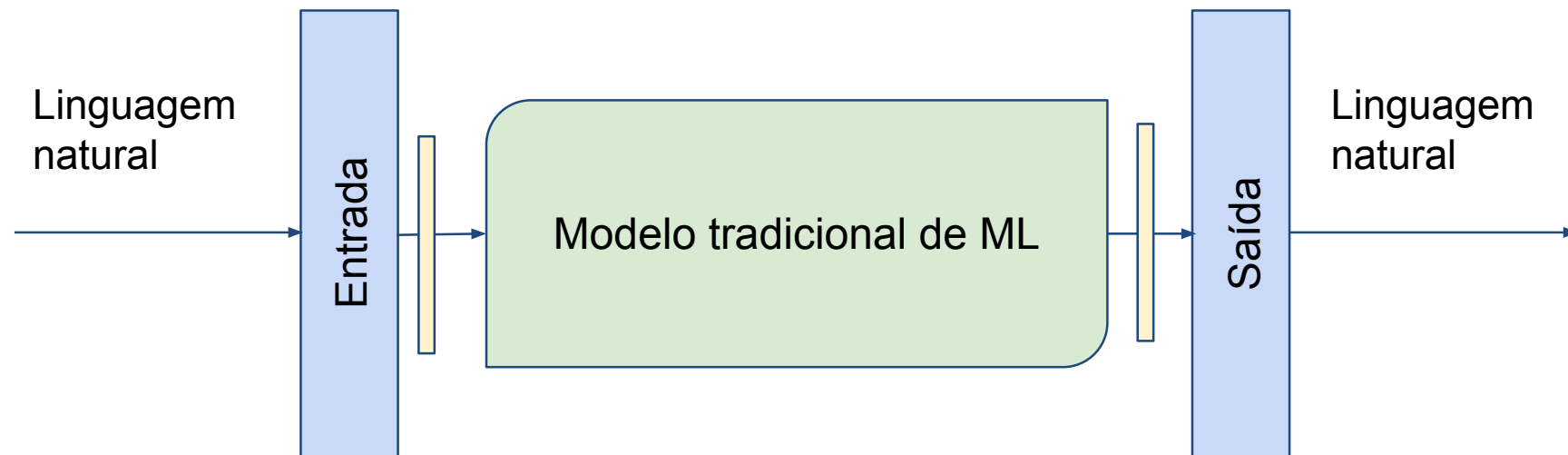
Contexto do ML "tradicional" no mundo da IA generativa

Durante o desenvolvimento



Contexto do ML "tradicional" no mundo da IA generativa

Inferência probabilística + determinística



Introdução ao módulo de ML

- Quais são os tipos de problemas / subdivisões de ML?

Introdução ao módulo de ML

- Quais são os tipos de problemas / subdivisões de ML?

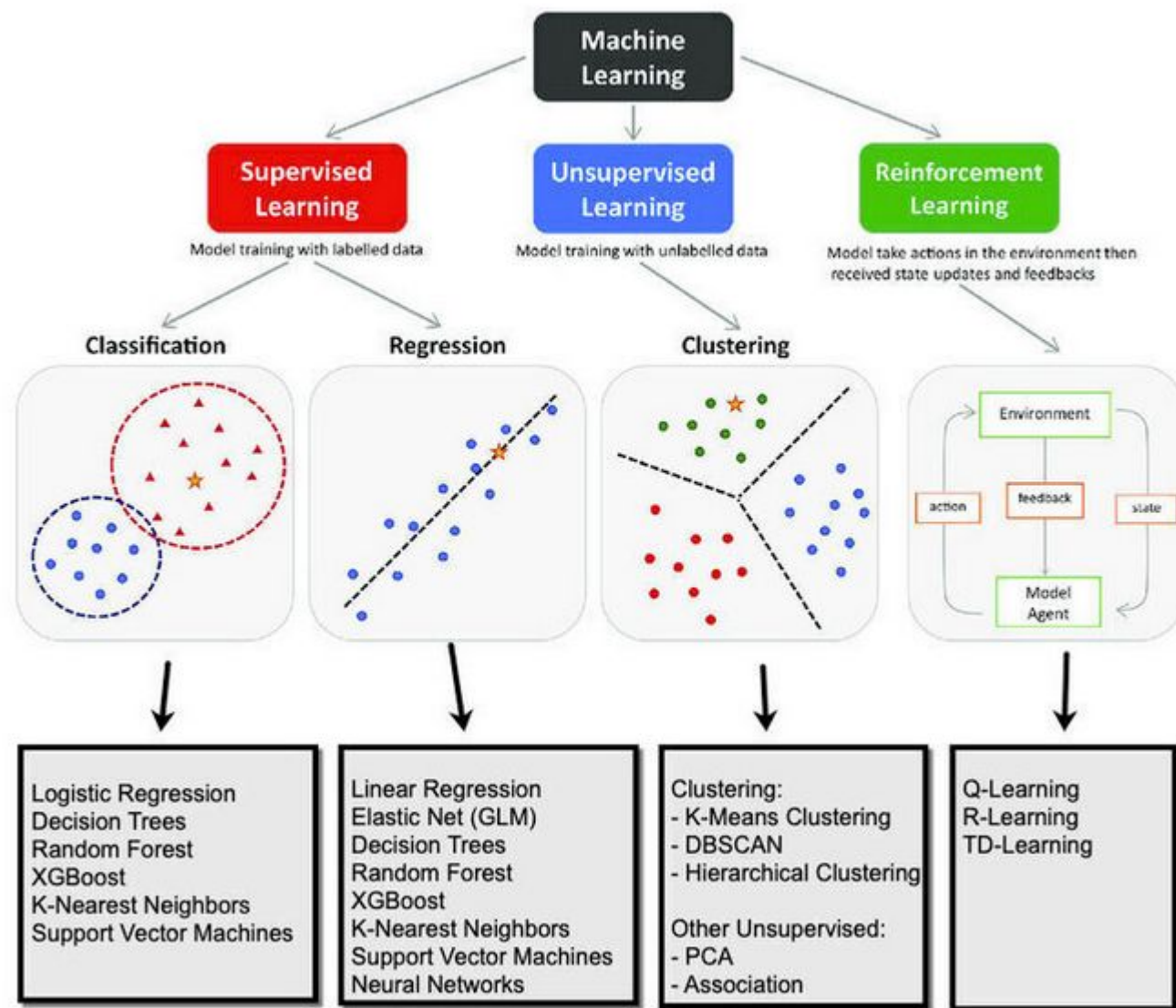
Relembrando IA I:

Há variável alvo/dependente categórica: **classificação**

Há variável alvo/dependente ordinal ou contínua: **regressão**

Não há variável alvo definida a priori: **agrupamento (clustering)**

Aprendizado por reforço



ML *lifecycle*

- Ciclo de ML: dados → treinamento → modelo → predição → avaliação



ML *lifecycle*

- Ver ML como parte de um processo mais amplo

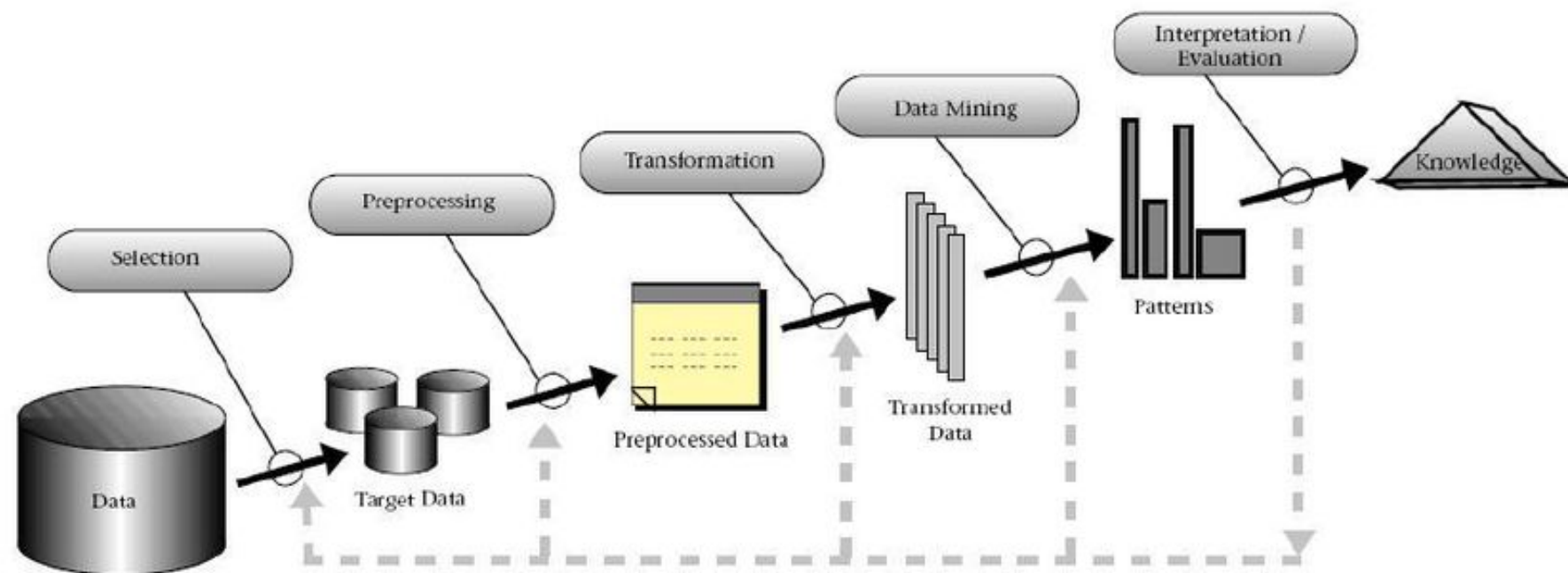


- Objetivo: transformar dados brutos → conhecimento útil

Knowledge Discovery in Databases (KDD)

- Processo sistemático de descoberta de padrões úteis em grandes volumes de dados.
- Vai além de algoritmos → envolve entendimento do domínio e interpretação dos resultados.
- Conceito surgido nos anos 90 (Fayyad et al., 1996).

Knowledge Discovery in Databases (KDD)



An overview of the steps that compose the knowledge discovery in databases (Fayyad et al. 1996)

Knowledge Discovery in Databases (KDD)

1. **Seleção:** escolher quais dados usar
2. **Pré-processamento:** limpar ruídos, tratar dados faltantes
3. **Transformação:** preparar atributos, normalizar, reduzir dimensionalidade
4. **Mineração de Dados:** aplicar algoritmos de ML (árvores, SVM, redes neurais etc.)
5. **Interpretação/Avaliação:** traduzir os resultados em conhecimento

ML é “só” a etapa 4. O sucesso depende do processo todo.

Modelos clássicos de ML

- Algoritmos tradicionais de ML focam em representações manuais dos dados.
- Exigem engenharia de atributos para extrair características relevantes.
- Mais interpretáveis, menos custo computacional.

Modelos clássicos de ML

- Algoritmos tradicionais de ML focam em representações manuais dos dados.
- Exigem engenharia de atributos para extrair características relevantes.
- Mais interpretáveis, menos custo computacional.

Tá... mas **o que isso quer dizer?** “representações manuais”,

“engenharia de atributo”, “extração de características”?

Modelos clássicos de ML

Em modelos tradicionais (árvores, SVM, regressão, etc.), o algoritmo não “enxerga” sozinho padrões complexos (como bordas numa imagem ou semântica em frases).

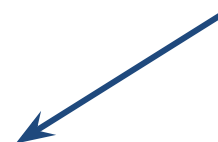
O cientista de dados precisa **criar atributos (features)** que representem bem o problema.

Ou seja, a **representação é manual** / artesanal, fruto de **conhecimento humano do domínio**.

Modelos clássicos de ML

Em modelos tradicionais (árvores, SVM, regressão, etc.), o algoritmo não “enxerga” sozinho padrões complexos (como bordas numa imagem ou semântica em frases).

E o que é isso
na prática?



O cientista de dados precisa **criar atributos (features)** que representem bem o problema.

Ou seja, a **representação é manual** / artesanal, fruto de **conhecimento humano do domínio**.

Atributos (*features*)

São **variáveis** ou **características** que descrevem cada instância (linha) do conjunto de dados.

O modelo de ML usa essas informações para aprender padrões.

Podem ser originais (já existentes nos dados) ou criadas pelo cientista de dados (engenharia de atributos).

Atributos (*features*)

Exemplo: Crédito bancário (dados tabulares)

Atributos: idade, renda mensal, valor de dívidas, número de parcelas atrasadas.

Tarefa: prever se o cliente vai pagar ou não um empréstimo.

Atributos (*features*)

Exemplo: Texto (classificação de spam)

Atributos: contagem de palavras (“promoção”, “grátis”), presença de links, número de caracteres em caixa alta.

Modelo usa esses atributos para prever se o e-mail é spam ou não.

Atributos (*features*)

Exemplo: Imagem de gato/cachorro (dados não estruturados)

Modelo clássico: precisa que você extraia manualmente **atributos**

→ cor média, número de bordas, tamanho de formas.

(***deep learning***: aprende automaticamente a reconhecer orelhas, olhos, focinho, etc. sem que você precise definir)

Reflexão

Se eu extraio atributos de uma imagem, isso vira dado tabular?

Reflexão

Se eu extraio atributos de uma imagem, isso vira dado tabular?

Sim, mas só depois da **transformação**.

Imagem original → matriz de pixels (não estruturado).

Se eu aplico um método de extração manual (ex.: bordas, cor média, número de objetos), eu transformo essa imagem em um vetor de atributos.

Esse vetor pode ser colocado em uma tabela (tabular) e usado em modelos clássicos.

Reflexão

Exemplo: Imagem de um gato

Original (não estruturado): 256x256 pixels (65 mil valores de cor).

Após feature engineering:

Nº de bordas = 123

Intensidade média da cor cinza = 0.62

Área do objeto principal = 75% da imagem

Isso vira uma linha em uma tabela.



Modelos clássicos de ML

São muito bons para **dados tabulares**.

Em dados tabulares (planilhas, bancos de dados estruturados), geralmente as variáveis (features) já têm significado direto.

- São eficientes mesmo com **datasets médios ou pequenos**.
- Costumam ter ótima **acurácia** em problemas tabulares.
- São mais **interpretáveis** (ex.: “clientes com renda $< X$ têm mais chance de inadimplir”).
- Exigem **menos custo** computacional.

Reflexão

E *deep learning*? É boa para esses casos (dados tabulares)?

Reflexão

Deep Learning, nesses casos:

Precisa de **muitos dados** para superar modelos clássicos.

Pode trazer **overfitting** em bases pequenas.

Muitas vezes não compensa o **custo** extra de treino/inferência.

Reflexão

IA Generativa (transformers, LLMs, diffusion models) brilha em dados **não estruturados**: texto, imagens, áudio, vídeo.

Para dados tabulares estruturados:

Modelos clássicos ainda são a escolha mais eficaz e simples.

Deep learning ou *Gen. AI* se justificam quando:

- Há **datasets gigantescos** (milhões de registros).
- Precisa-se capturar **interações muito complexas** entre variáveis.
- O problema envolve **multimodalidade** (ex.: misturar tabelas + texto livre + imagens).

Modelos clássicos de ML

Vamos ver alguns algoritmos:

- KNN
- Decision trees
- SVM
- ...

K-Nearest Neighbors (KNN)

Intuição:

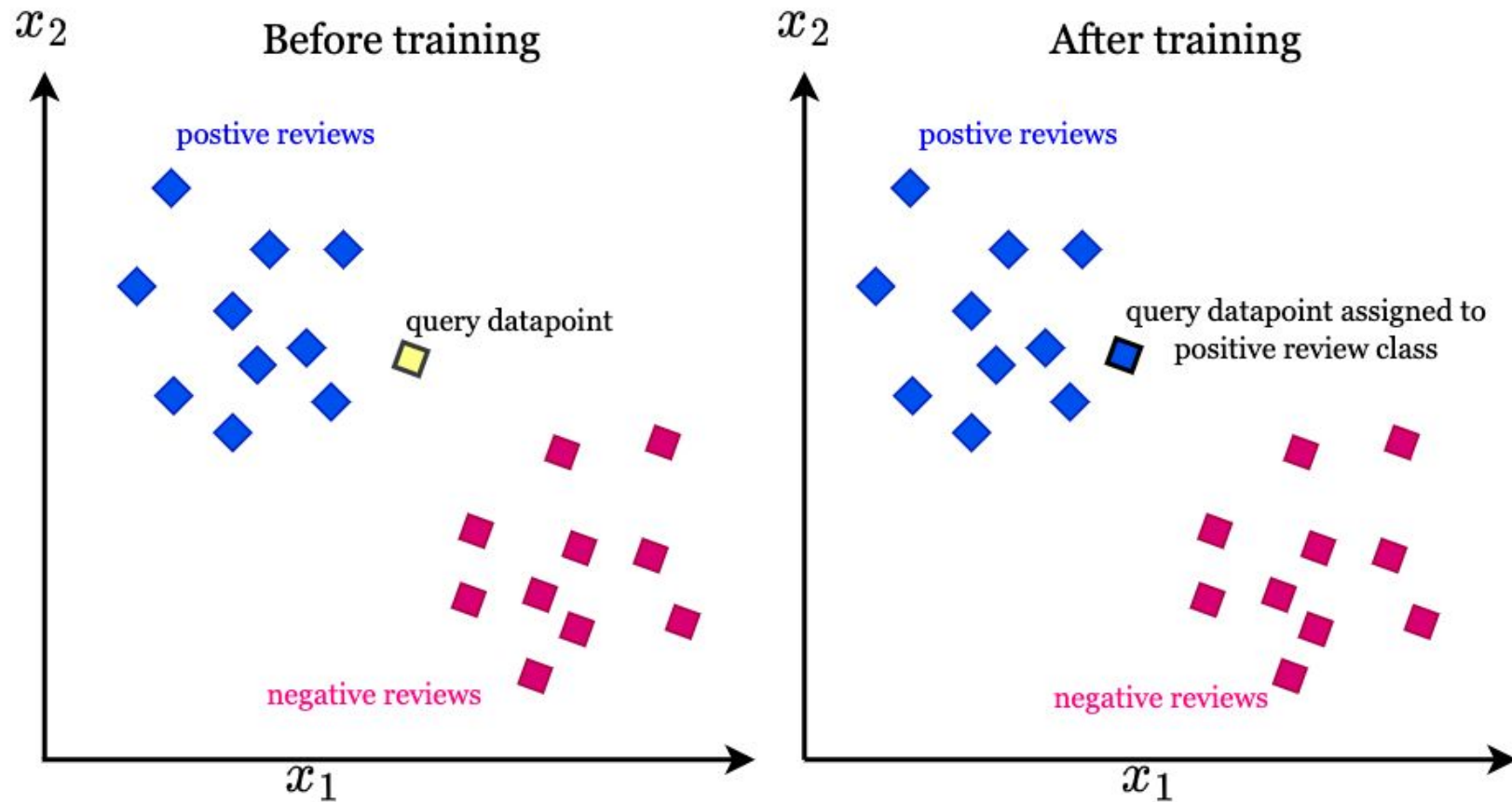
“Diz-me quem são teus vizinhos e te direi quem és.”

Não treina um modelo explícito; guarda os dados e decide por votação (classificação) ou média (regressão).

Sensível à escala (normalizar!), custo de predição cresce com N .

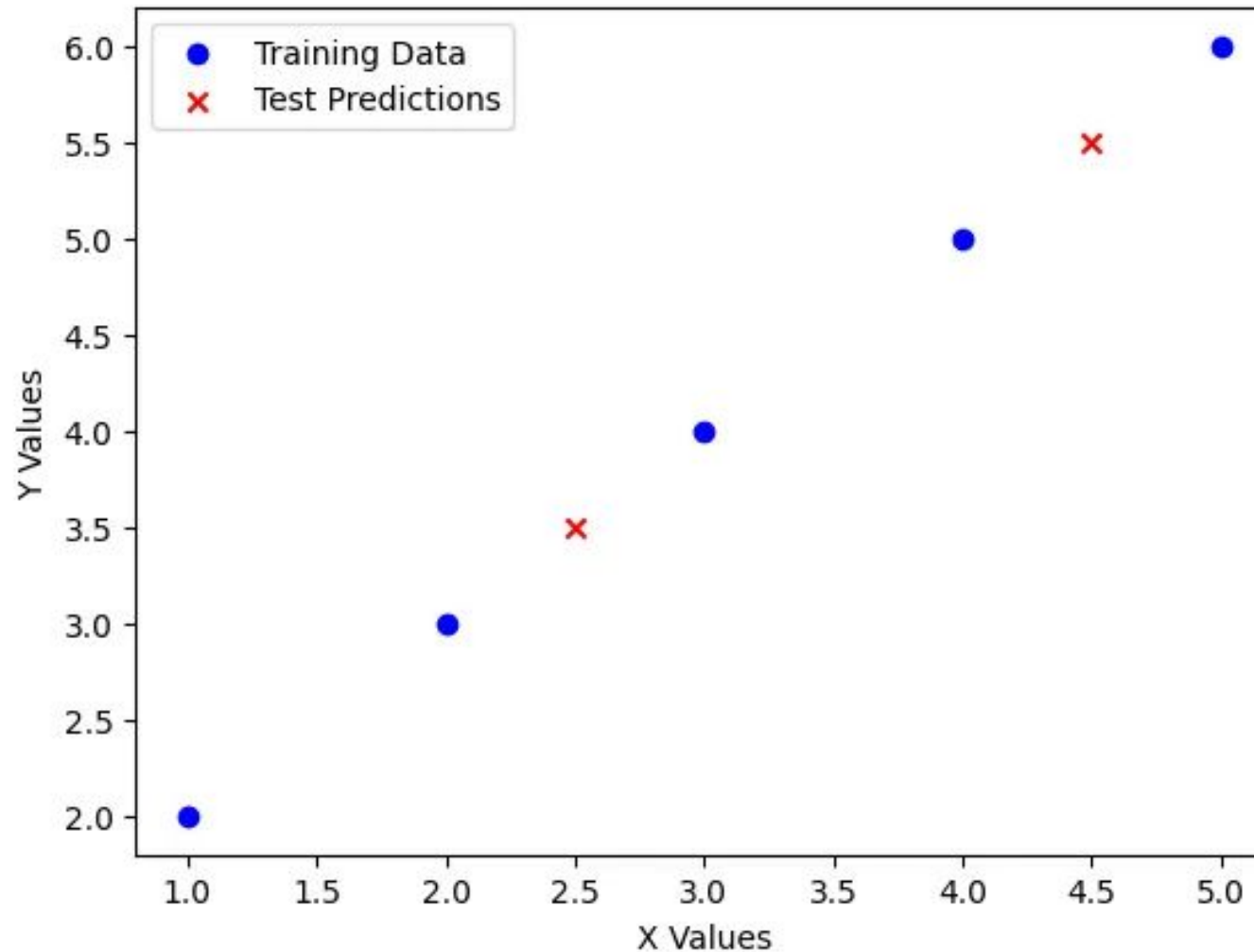
K-Nearest Neighbors (KNN)

Classificação:



K-Nearest Neighbors (KNN)

Regressão (KNN *Regression*):



K-Nearest Neighbors (KNN)

Hiperparâmetros chave: *n_neighbors*, *weights* (uniform/distance),
métrica (euclidiana, Manhattan).

Quando usar: *datasets* pequenos/médios, fronteiras não lineares,
baseline rápido.

Atenção: alto custo na inferência, sofre em alta dimensionalidade.

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

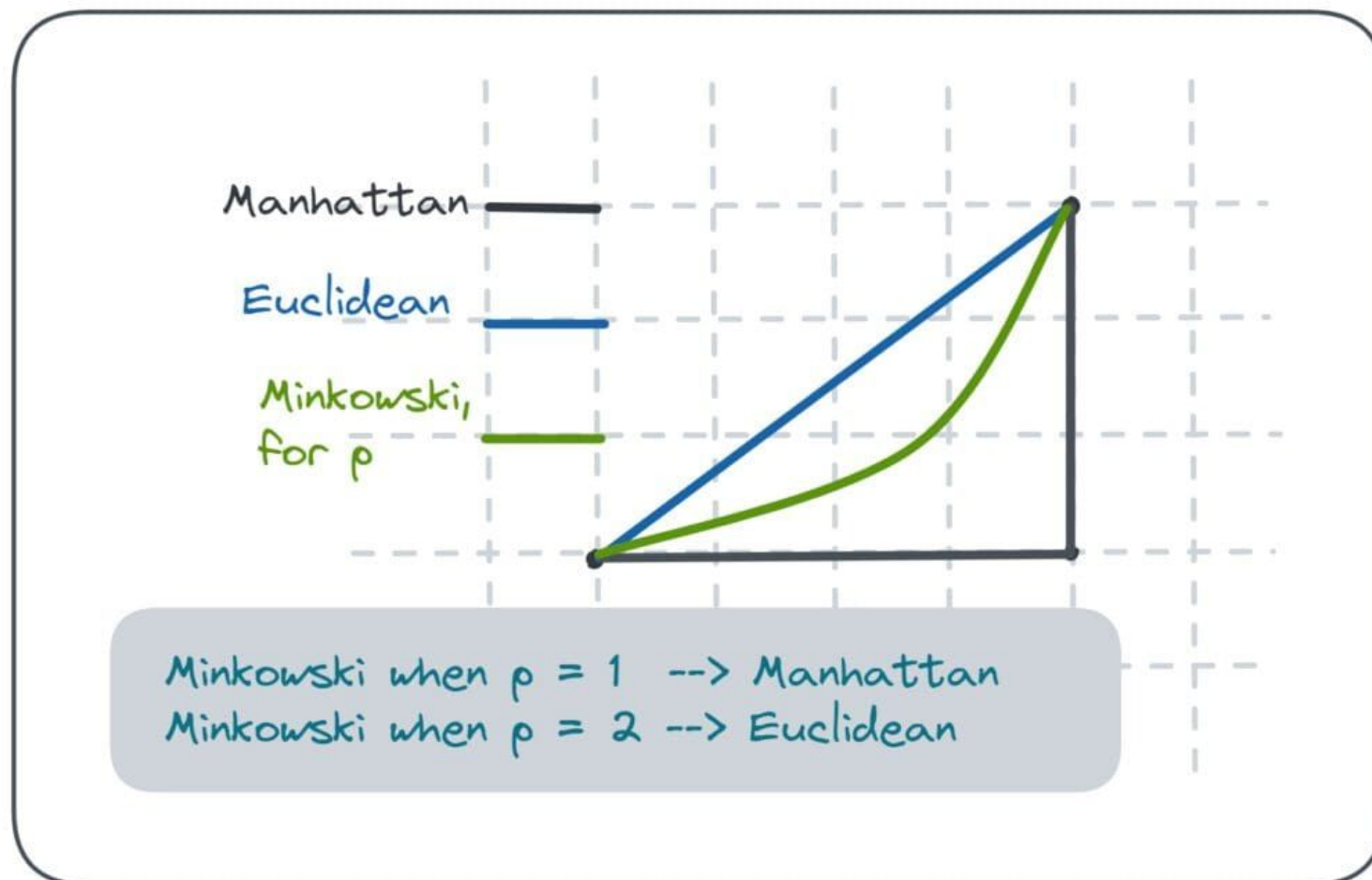
Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

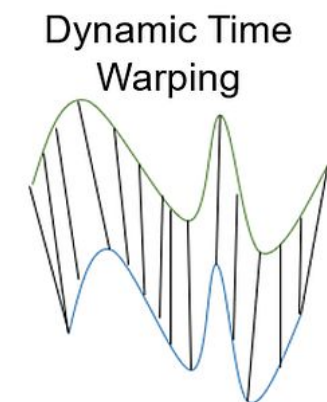
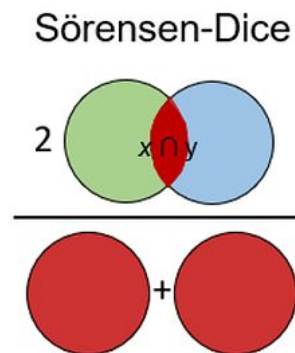
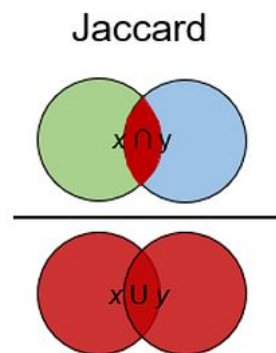
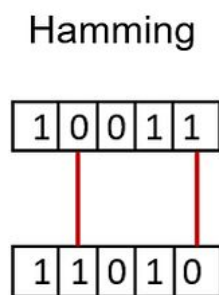
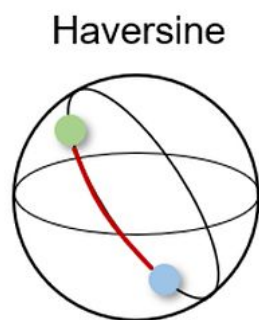
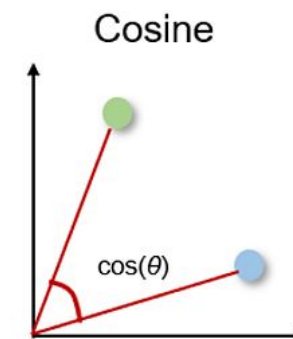
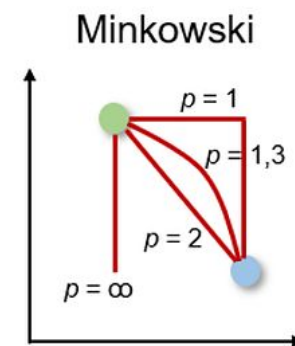
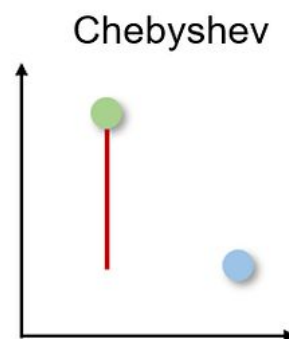
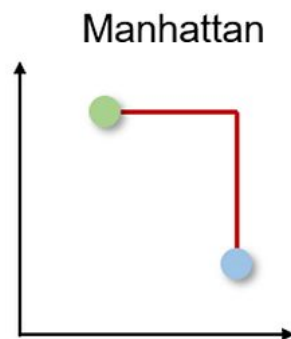
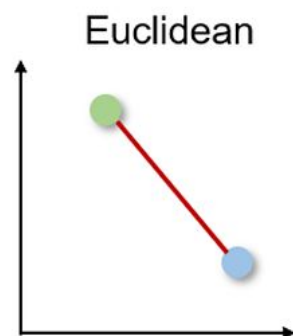
Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

Distance functions



Distance functions



Prática KNN

(Jogo, Horas de história, Intensidade da ação, Classe)

```
# Dataset manual
jogos = [
    ("Skyrim", 100, 20, "RPG"),
    ("The Witcher", 120, 15, "RPG"),
    ("Doom", 10, 95, "FPS"),
    ("CS:GO", 5, 90, "FPS"),
    ("Mass Effect", 80, 40, "RPG"),
    ("Call of Duty", 12, 85, "FPS")
]
```

Prática KNN

(Jogo, Horas de história, Intensidade da ação, Classe)

Quantas **dimensões** tem esse *dataset*?

Dimensão = cada característica

Prática KNN

Features (dimensões do espaço de atributos):

- Horas de história
- Intensidade da ação

Target (rótulo / classe a ser prevista):

- Classe (RPG ou FPS).

Nome do jogo → aqui é só um identificador, não serve como *feature*.

Prática KNN

Source: `1_knn.py`

Exemplo: se chega um novo jogo, "Novo Game" com 50h história, 50 de ação.

```
novo_jogo = ("Novo Game", 50, 50)
```

Calcular distância dele para todos os outros pontos.

Escolher k vizinhos mais próximos.

Dimensão = 2

Distância = como medir a semelhança entre objetos (vamos usar euclidiana).

K = número de vizinhos, importância da escolha (vamos usar 3).

Classificação = a classe é definida por “quem está mais perto”.

Reflexão

KNN é um algoritmo de ML supervisionado, dentro da área de IA.

Ele “aprende” apenas armazenando os dados de treino e, depois, usando distâncias para classificar um novo ponto.

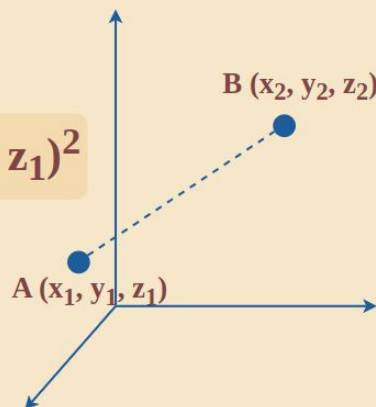
A decisão final (RPG ou FPS) **não foi programada por você diretamente** → foi “deduzida” (estatisticamente) pelo algoritmo a partir do conjunto de exemplos.

Ou seja: **não é uma regra fixa** feita pelo programador (“se horas > 50, então RPG”), mas sim um **modelo baseado em dados**.

E se tiver mais dimensões?

Distance Between Two Points in 3D

$$AB = (x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2$$



Distance with four predictors

	Age	Inc	Work	Fam
Cust ₁	43	68	11	5
Cust ₂	40	64	16	3

$$\begin{aligned}d &= \sqrt{(a_1 - a_2)^2 + (i_1 - i_2)^2 + (w_1 - w_2)^2 + (f_1 - f_2)^2} \\&= \sqrt{(43 - 40)^2 + (68 - 64)^2 + (11 - 16)^2 + (5 - 3)^2} \\&= \sqrt{9 + 16 + 25 + 4} \\&= \sqrt{54} = 7.35\end{aligned}$$

Note: in this example, the variables are not standardized

Normalização

KNN é **sensível à escala das variáveis**.

No nosso dataset, a *feature* Horas de história vai de 0 a 120, enquanto Intensidade da Ação vai de 0 a 100.

Se tivesse uma variável em outra escala (ex: preço de jogo em R\$, indo até milhares), ela dominaria o cálculo da distância.

Por isso, a **normalização** (ou padronização) é fundamental.

Normalização

Sem normalizar: atributos com números grandes pesam mais no cálculo da distância.

Com normalização: todos os atributos ficam numa mesma escala (ex: 0 a 1).

Min-Max Scaling:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Existem outros tipos: normalização Z-score, logarítmica, etc.

Problema da alta dimensionalidade

Para classificar um ponto novo, o KNN precisa calcular a distância dele para **todos os pontos do dataset**.

Cada distância envolve somar diferenças em **todas as dimensões**.

Se temos:

N = número de pontos no dataset,

D = número de dimensões (features),

então o custo é aproximadamente:

$$O(N \cdot D)$$

Ou seja: mais dimensões → mais contas por ponto → algoritmo mais lento (**custo computacional**).

Problema da alta dimensionalidade

Além do custo computacional, tem um **problema conceitual**:

Quando aumentamos o número de dimensões, os **pontos ficam mais esparsos**.

A distância entre pontos começa a ficar “parecida”: todo mundo parece igualmente distante.

Isso dificulta a noção de “vizinhos próximos”, que é a base do KNN.

Prática KNN

- Ver `2_knn.py` (exemplo para prever rating de filmes de usuários)
- Ver `3_knn.py` (exemplo de KNN com imagens)

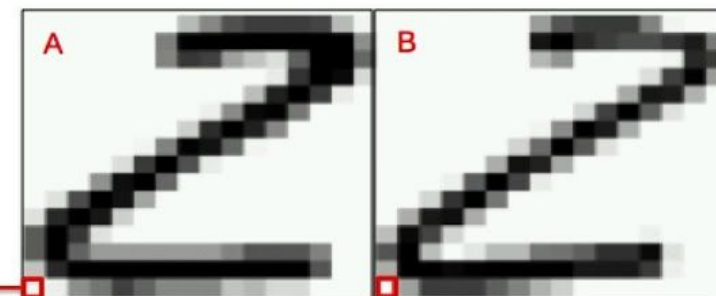
Example: handwritten digits

- 16x16 bitmaps
- 8-bit grayscale
- Euclidian distance

– over raw pixels

$$D(A,B) = \sqrt{\sum_r \sum_c (A_{r,c} - B_{r,c})^2}$$

- Accuracy:
 - 7-NN ~ 95.2%
 - SVM ~ 95.8%
 - humans ~ 97.5%



Árvores de Decisão (*decision trees*)

Intuição:

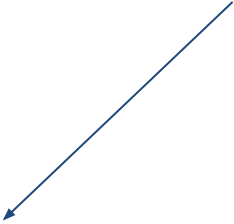
Regras “if-else” que particionam o espaço/dividem os dados (e.g., “se renda<... e idade>...”).

Muito interpretável; bom para tabulares; propenso a *overfitting* se profundo.

Aplicações: diagnóstico médico, classificação de clientes, risco de crédito.

Árvores de Decisão (*decision trees*)

- **Hiperparâmetros chave:** max_depth, min_samples_split, min_samples_leaf, criterion (gini/entropy).
- **Quando usar:** explicabilidade, tabulares, baseline rápido.
- **Atenção:** instável a pequenas variações (use *ensembles*), pode superajustar.

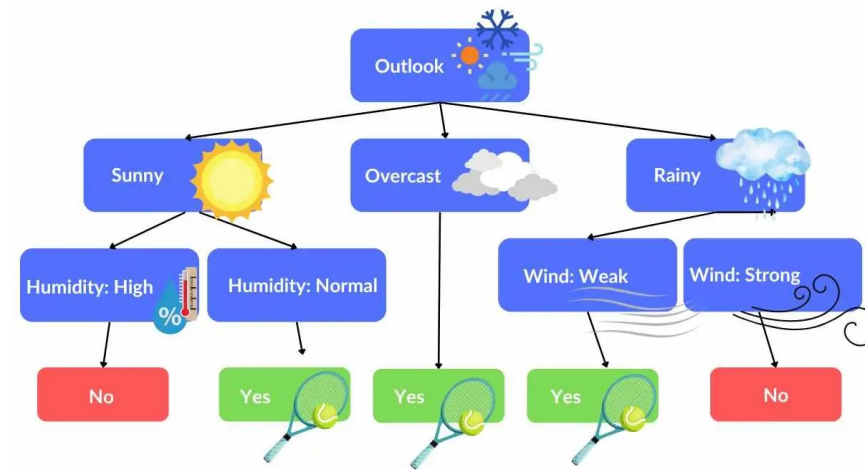


Métodos que combinam várias árvores de decisão (ou outros modelos) para produzir um resultado mais estável e robusto.

Vamos ver sobre **Random Forest** e **Boosting** mais para frente na disciplina!

Reflexão

- Em essência: árvore de decisão é só uma **árvore de if-else**
- Cada nó guarda:
 - feature usada para o split (ex.: "horas de história"),
 - limiar de corte (ex.: < 50),
 - ponteiros para os filhos (esquerda/direita).
- No final, cada folha guarda a classe.



Então, onde entra o “ML”?

Reflexão

A parte “difícil” não é rodar a árvore (isso é “só” if-else), mas sim **construí-la automaticamente a partir dos dados.**

Isso exige algoritmos que definem:

- Qual *feature* escolher para o split em cada nó.

- Qual ponto de corte usar (ex.: "idade < 30").

- Quando parar de dividir (parada por profundidade, pureza etc.).

Prática *Decision Trees*

**Classificar se uma pessoa joga videogame (sim/não),
baseado em idade e tempo livre por semana**

```
# Dataset manual
data = {
    "idade": [15, 25, 32, 40, 50, 65, 12, 22, 28, 35],
    "horas_livres": [30, 10, 5, 8, 2, 1, 25, 15, 7, 4],
    "joga": ["sim", "sim", "não", "não", "não", "não", "sim", "sim", "não", "não"]
}
```

Source: `1_dtrees.py`

Reflexão

- Se na sua execução a variável `horas_livres` não aparece na árvore, provavelmente a árvore decidiu que só `idade` já é suficiente para separar o `target` (joga). Isso acontece porque:
 - Dataset pequeno: A árvore procura o **melhor *split*** e pode ter encontrado divisões que já classificam 100% dos dados só olhando a `idade`.
 - **Critério de pureza** (*entropy* ou *gini*): se um atributo explica melhor os rótulos, os outros ficam de lado.
 - **Correlações fortes**: no dataset de exemplo, quem tem menos `idade` tende a jogar, quem tem mais `idade` tende a não jogar → a árvore “ignora” `horas_livres`.
- Ou seja: o algoritmo não “força” o uso de todas as *features*, ele só usa o que ajuda a **diminuir a entropia**.

Reflexão

- A árvore escolhe os melhores cortes baseados em métricas de **pureza**.
- Duas métricas comuns:
 - **Entropia** (quando `criterion="entropy"`)
 - **Gini** (default)

Tanto **Entropia** quanto **Gini** medem a **impureza** de um nó:

Quanto mais “misturada” a classe está → maior o valor.

Quanto mais homogênea (pura) → valor próximo de **0**.

A árvore sempre tenta **reduzir essa impureza** a cada split.

Reflexão

O que significa "diminuir entropia"?

O que é **entropia**?

Fórmula da entropia (teoria da informação)

$$H = - \sum p(x) \log p(x)$$

Onde:

$p(c)$ = probabilidade de cada classe no conjunto.

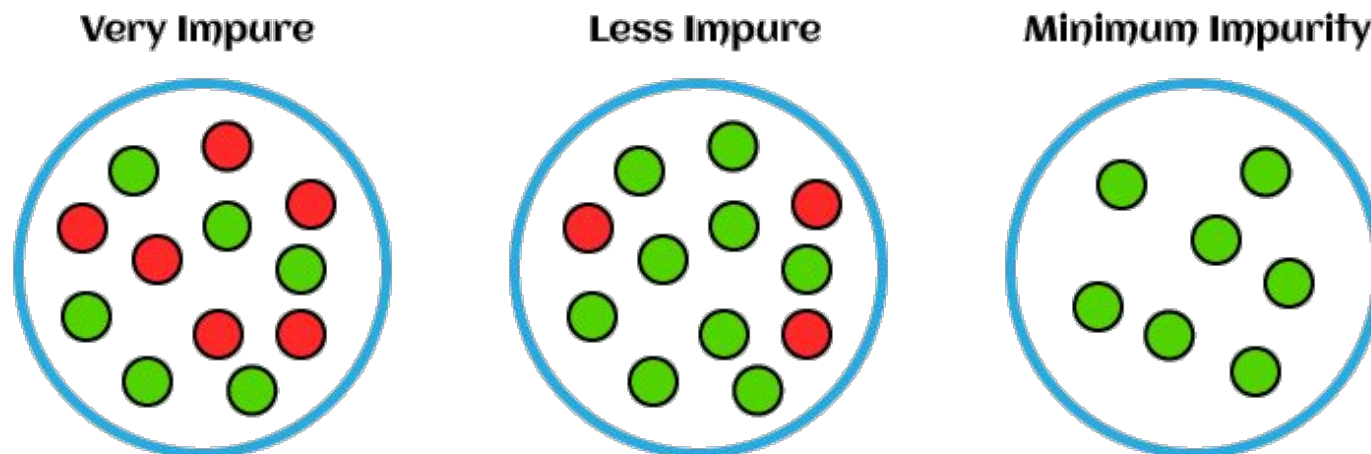
O logaritmo em base 2 é usado porque estamos medindo em bits de informação.

Reflexão

De forma bem simples, entropia é uma **medida de incerteza ou desordem** de um conjunto de dados.

Se todos os elementos são iguais $\rightarrow$ não há incerteza $\rightarrow$ entropia = 0.

Se os elementos estão bem misturados (ex.: metade de cada classe) $\rightarrow$ máxima incerteza $\rightarrow$ entropia = alta.



Reflexão

Caixa A: 10 bolas todas vermelhas → você tem 100% de certeza que, ao pegar uma bola, ela será vermelha.

Entropia = ?

Reflexão

Caixa A: 10 bolas todas vermelhas → você tem 100% de certeza que, ao pegar uma bola, ela será vermelha.

Entropia = 0 (nenhuma incerteza).

Reflexão

Caixa B: 5 vermelhas e 5 azuis → você tem 50% de chance de tirar cada cor.

Entropia = ?

Reflexão

Caixa B: 5 vermelhas e 5 azuis → você tem 50% de chance de tirar cada cor.

Entropia = máxima (maior incerteza possível nesse caso).

Reflexão

Caixa C: 9 vermelhas e 1 azul $\rightarrow$ ainda há incerteza, mas bem menor do que a da Caixa B.

Entropia = intermediária.

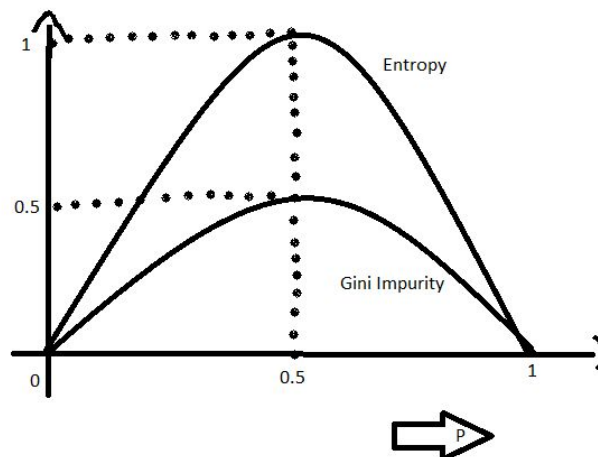
Reflexão

- O índice de **Gini** mede a impureza de um nó: quão misturadas estão as classes naquele conjunto.
 - Se todas as amostras são da mesma classe $\rightarrow$ Gini = 0 (nó puro).
 - Quanto mais misturado (mais classes balanceadas) $\rightarrow$ maior o Gini.

Entropia: Usa logaritmos $\rightarrow$ cálculo mais pesado. Às vezes gera árvores um pouco mais profundas e balanceadas.

Gini: Mais simples (sem log). Mede “probabilidade de classificar errado se escolher aleatoriamente de acordo com as proporções”.

Reflexão



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

- Entropia cresce um pouco mais rápido perto do equilíbrio (50/50).
- Gini é mais linear.
- Ambos têm a mesma forma qualitativa (zero nos extremos, máximo no meio), mas escalas diferentes.
- Gini e entropia **não são múltiplos fixos** um do outro.
- Só coincidem aproximadamente no caso mais incerto (50/50).

Prática *Decision Trees*

Source: `2_dtrees.py`

- A árvore escolhe, de forma gulosa, o atributo que mais reduz a impureza em cada nó. Se o conjunto de exemplos “premia” horas_livres (ele sozinho já separa quase tudo), a árvore vai usá-lo e pode ignorar idade.
- Para garantir que as duas features entrem, podemos usar um *dataset* com uma **regra que depende de ambos**:

Se $\text{idade} < 30$ precisa de ≥ 10 horas livres para “joga = sim”.

Se $\text{idade} \geq 30$ precisa de ≥ 20 horas livres para “joga = sim”.

Reflexão

DecisionTreeClassifier do *scikit-learn* tem alguns pontos de **aleatoriedade**:

Divisão de nós quando há empate: se dois *splits* dão exatamente o mesmo ganho de informação, o algoritmo pode escolher qualquer um.

Parâmetros que usam amostragem: se usar `splitter="random"` ou permitir $max_features < n_features$, ele vai sortear subconjuntos de atributos.

Critério de parada: quando há mais de uma forma igualmente boa de dividir, a escolha pode mudar de execução pra execução.

Para ter **reprodutibilidade**, usamos `random_state=42` (ou qualquer número fixo). Isso garante que, dado o mesmo dataset, a árvore será sempre igual.

Support Vector Machines (SVM)

Intuição:

- Encontra o **hiperplano com maior margem entre classes**.
- Com *kernel* (RBF, polinomial) mapeia para espaços de maior dimensão e separa casos não lineares.
- Bom em alta dimensionalidade com N moderado.

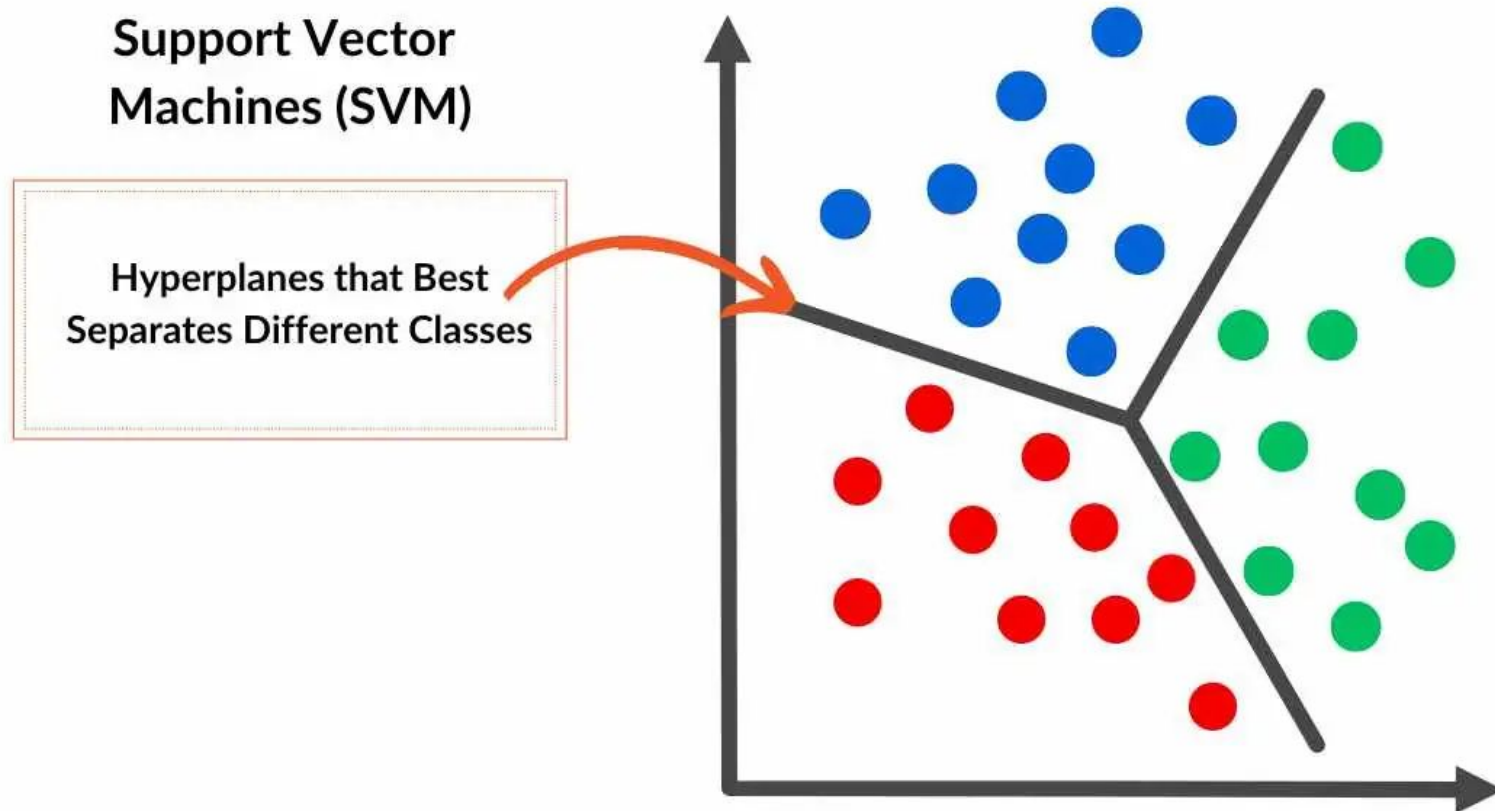
Support Vector Machines (SVM)

Intuição:

????

- Encontra o **hiperplano com maior margem entre classes**.
- Com *kernel* (RBF, polinomial) mapeia para espaços de maior dimensão e separa casos não lineares.
- Bom em alta dimensionalidade com N moderado.

Support Vector Machines (SVM)

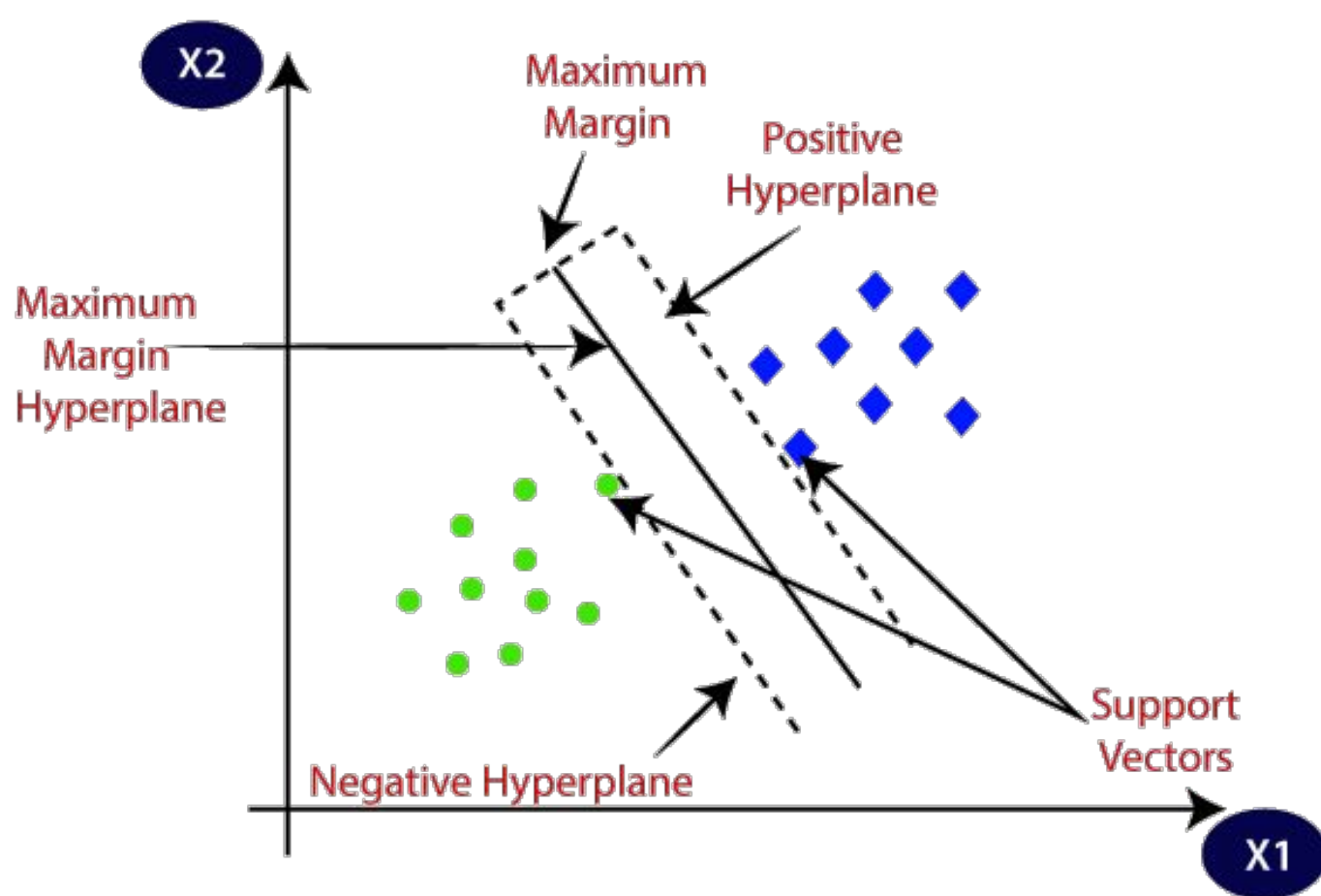


2D (2 features) → o hiperplano é uma reta.

3D (3 features) → o hiperplano é um plano (entidade geométrica bidimensional).

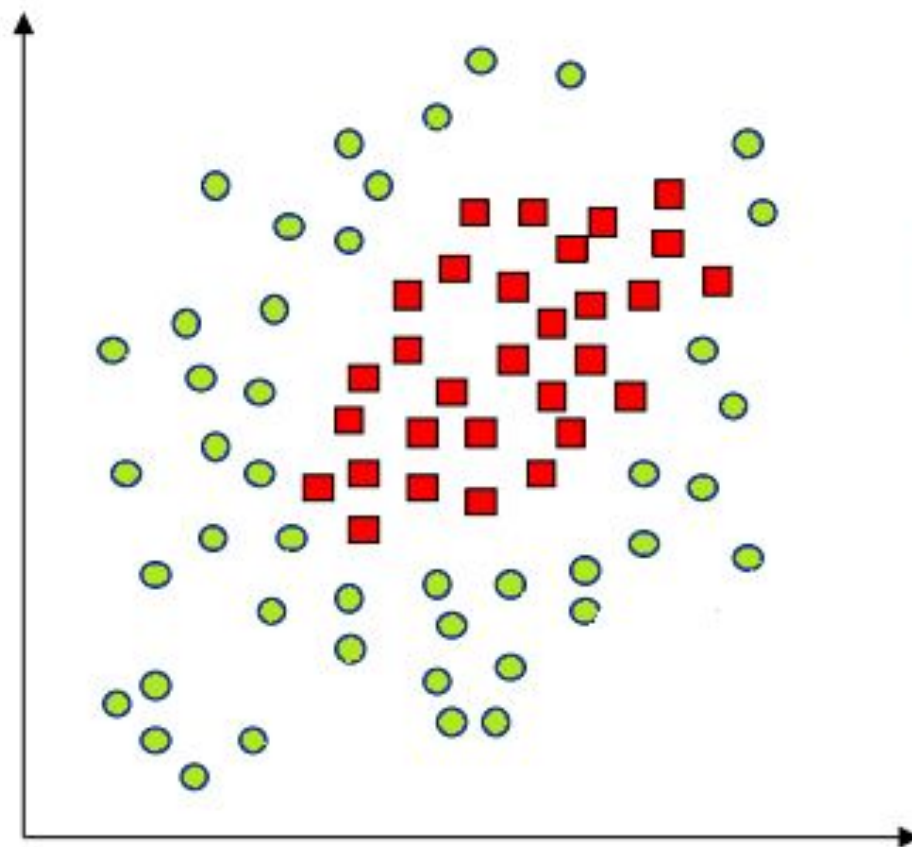
N dimensões → chamamos genericamente de hiperplano.

Support Vector Machines (SVM)



Support Vector Machines (SVM)

Muitas vezes, os dados não são linearmente separáveis (não dá pra traçar uma linha reta que divide bem as classes).

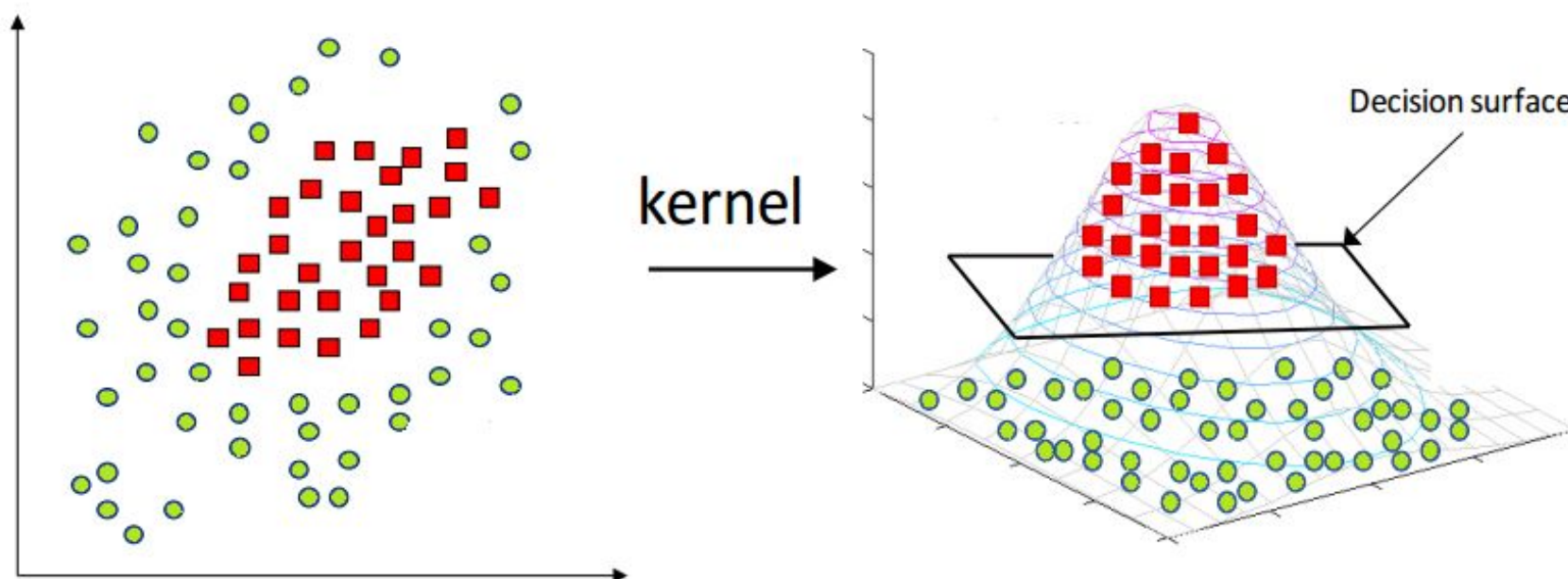


Será que dá para usar SVM?

Support Vector Machines (SVM)

Kernel é um “truque matemático” que projeta os dados em um espaço de maior dimensão, onde pode existir um hiperplano linear que os separa.

Com esse *kernel trick*, não precisamos calcular explicitamente todos os atributos. O próprio algoritmo SVM aplica o kernel automaticamente durante o treinamento.



Support Vector Machines (SVM)

- **Hiperparâmetros chave:** C (penalidade), kernel (linear/RBF/poly), gamma (RBF).



Radial Basis Function,
também chamado de Gaussiano

- **Quando usar:** textos/vetores esparsos, margens claras, *datasets* não gigantes.
- **Atenção:** escala mal para N muito grande; exige normalização; tuning de C/gamma.

Prática SVMs

Source: `1_svm.py`

(nome, calorias, tempero[0-5], classe)

```
comidas = [  
    ("Hambúrguer", 500, 3, "Lanche"),  
    ("Pizza",      700, 4, "Lanche"),  
    ("Coxinha",    300, 3, "Lanche"),  
    ("Pastel",     450, 4, "Lanche"),  
  
    ("Feijoada",   900, 5, "Prato pesado"),  
    ("Lasanha",    800, 3, "Prato pesado"),  
    ("Churrasco",  850, 4, "Prato pesado"),  
    ("Strogonoff", 750, 2, "Prato pesado"),  
  
    ("Sorvete",    250, 1, "Sobremesa"),  
    ("Bolo",       400, 2, "Sobremesa"),  
    ("Pudim",     350, 1, "Sobremesa"),  
    ("Mousse",    300, 1, "Sobremesa"),  
]
```

Reflexão

Vamos analisar como os hiperparâmetros do SVM moldam o limite de decisão.

Perceba que com o classificador atual, Pizza foi classificada com Prato pesado ao invés de Lanche.

C (*regularization/penalty parameter*): diz ao SVM o quanto penalizar erros no treino. Seria o peso da punição” para cada erro.

γ , parâmetro do *kernel* RBF): Controla a influência de cada ponto de treino. Seria o “raio de influência” de cada ponto.

Reflexão

`SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42)`

C = 1.0 → **penalidade pequena para erros**. O modelo aceita classificar alguns pontos errado em troca de ter uma fronteira de decisão mais suave e ampla.

gamma = "scale" → valor calculado automaticamente (geralmente baixo). Isso faz com que cada ponto de treino tenha uma influência larga, **difusa**. O resultado é uma fronteira suave, que não se ajusta demais aos dados.

Como a Pizza (700 kcal) está mais próxima das calorias dos “Pratos pesados” (750–900 kcal) do que dos lanches (300–500 kcal), o modelo “puxa” ela para o lado errado.

Reflexão

```
SVC(kernel="rbf", C=100.0, gamma=1.0, random_state=42)
```

C = 100.0 → **penalidade muito maior para erros**. O modelo tenta classificar todos os pontos de treino corretamente, mesmo que precise criar **fronteiras mais complexas**.

gamma = 1.0 → bem maior que o valor automático. Faz com que cada ponto de treino tenha uma influência mais local. A fronteira de decisão fica mais “apertada” ao redor dos pontos.

Reflexão

Em um conjunto pequeno e “limpo” (atual: 12 pontos bem definidos), isso parece bom, porque o modelo classifica tudo certo e não sofre com ruído.

Mas em dados reais (grandes, com outliers e variação natural), isso provavelmente levaria a **overfitting**, ou seja, um modelo que acerta no treino mas erra quando aparece algo novo.

Em resumo:

C e gamma baixos → modelo mais suave, pode errar no treino, mas tende a **generalizar** melhor.

C e gamma altos → modelo **superajustado** ao treino, ótimo naquele conjunto, mas arriscado fora dele.

Prática SVMs

- Ver `2_svm.py`

Função `make_moons` da biblioteca *scikit-learn*.

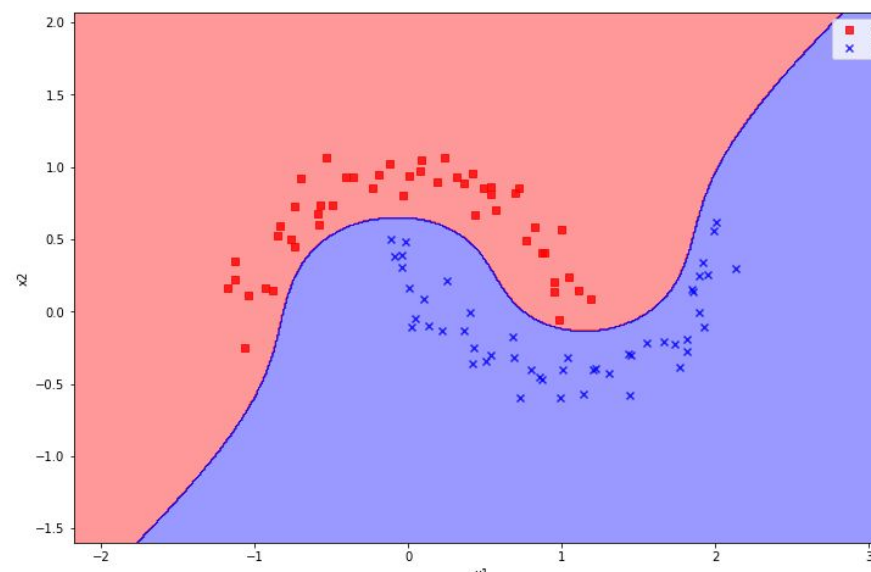
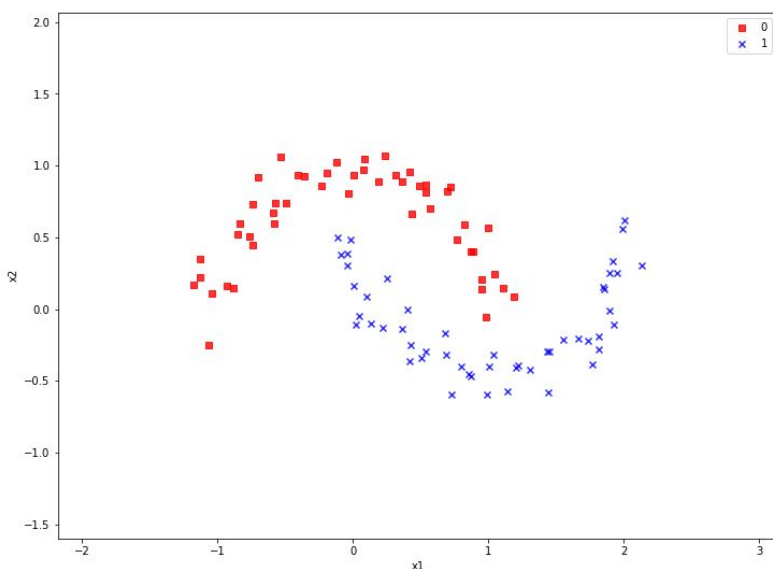
Formato:

2 classes em formato de meias-luas entrelaçadas.

Cada ponto tem 2 atributos (coordenadas x e y).

Objetivo: Classificar os pontos em Classe 0 ou Classe 1.

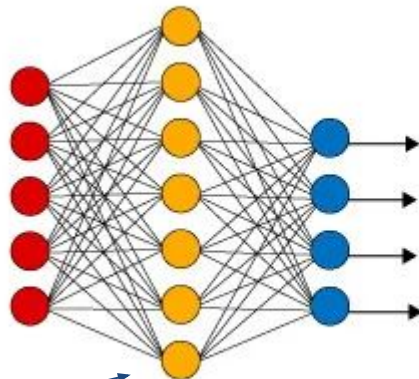
Desafio: As classes não são linearmente separáveis (não dá para dividir com uma linha reta).



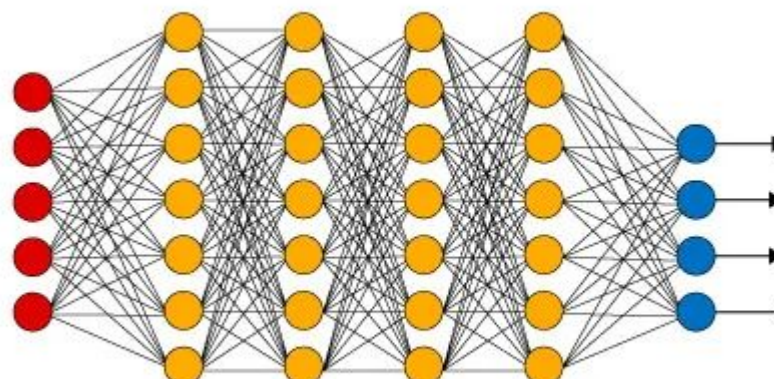
Redes neurais e aprendizado profundo

- Inspiradas em redes neurais biológicas.
- Processam dados em camadas de **neurônios artificiais**.
- Aprendem representações automáticas (menos engenharia manual).
- Necessitam grandes volumes de dados e alto poder computacional.

Simple Neural Network



Deep Learning Neural Network



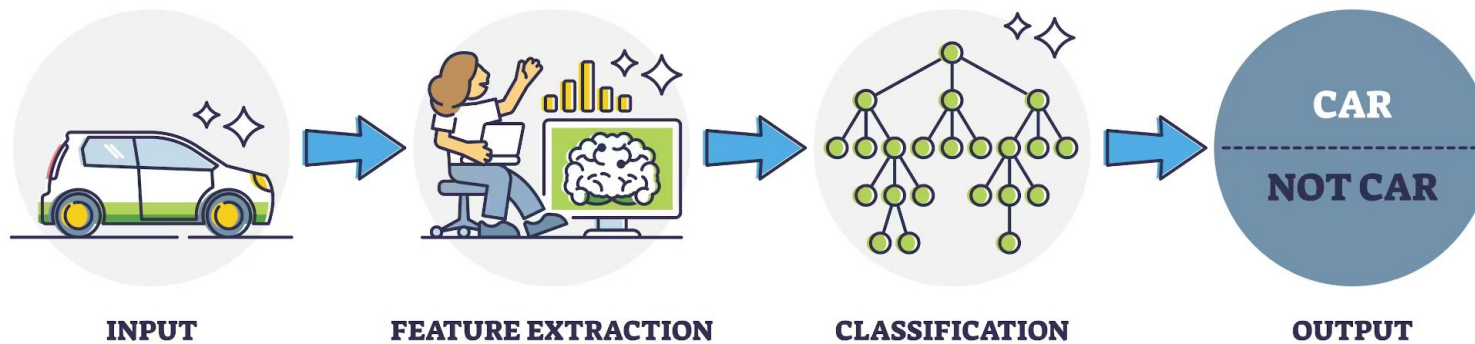
● Input Layer

● Hidden Layer

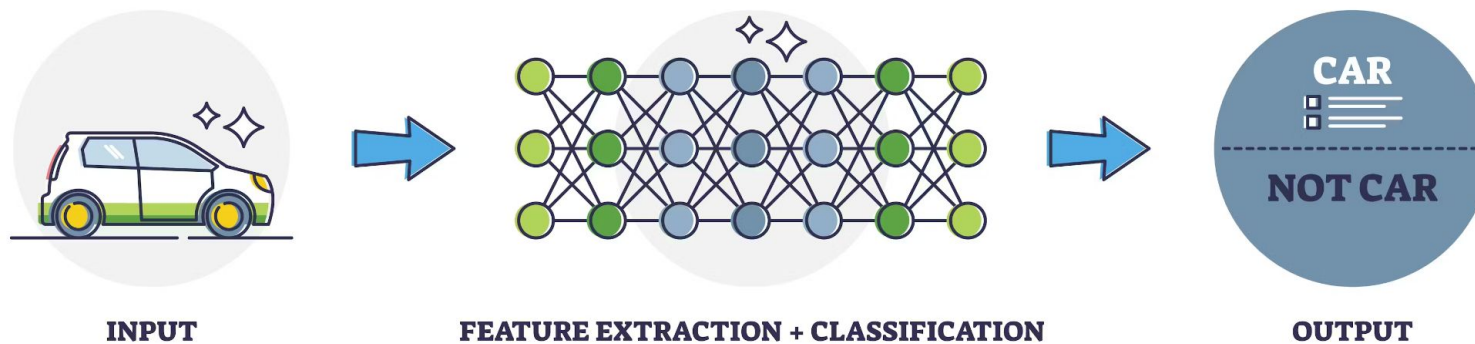
● Output Layer

Redes neurais e aprendizado profundo

MACHINE LEARNING



DEEP LEARNING



- Vamos voltar a falar de NN e DL em aulas futuras.

Redes neurais e aprendizado profundo

- Lembre que **DL é um subgrupo de NN**.
 - Uma *shallow neural network* (rede neural rasa), como o perceptron multicamada com 1 camada escondida, já existiam antes do DL. Elas conseguem aprender representações não lineares, mas têm capacidade limitada se comparadas às redes profundas.
 - Em geral, exigem mais recursos que ML clássico, mas menos que DL.
- DL é caracterizado por **muitas camadas** → **maior capacidade de abstração**, mas também **maior custo**.

O que levar desse módulo...

- **Modelos clássicos** são úteis e eficientes em muitos contextos:
 - Muito eficazes em **dados tabulares e estruturados** (ex.: planilhas, bases de clientes);
 - Mais **interpretáveis** e **rápidos** em contextos estruturados;
 - Demandam **menos recursos** computacionais e dados para atingir boa performance.
- **Redes profundas** conseguem superar em tarefas complexas:
 - **Muitas camadas** → extraem **representações complexas**.
 - Excelentes para **dados complexos e não estruturados** (imagens, áudio, texto em larga escala).
 - Exigem **mais recursos**: grandes volumes de dados, maior poder computacional e mais tempo de ajuste.

O que levar desse módulo...

- ML clássico continua relevante:
 - Em muitos problemas, ele ainda supera ou iguala o *deep learning*.
 - Em geral, deve ser a primeira escolha antes de recorrer a soluções mais pesadas.
- O segredo está em escolher o **modelo certo para o problema certo**.

Próxima aula...

- Prática com pré-processamento de dados.
- Aprofundaremos em engenharia de atributos.